Implementation and Analysis of ExpressLRS Under Interference Using GNU Radio

Gabriel Garcia GARCIAG2022@FAU.EDU
Georgios Sklivanitis GSKLIVANITIS@FAU.EDU
Dimitris A. Pados DPADOS@FAU.EDU
Center for Connected Autonomy and AI (CA-AI.fau.edu), Florida Atlantic University, Boca Raton, FL, USA

Abstract

ExpressLRS is an open-source high-refresh radio control link that has proven resilient to interference while maintaining a maximum achievable range at that rate with low latency. Designed to be the best FPV Racing link, it is based on the Semtech SX127x/SX1280 LoRa hardware combined with an Espressif or STM32 processor. We build the first known open-source implementation of ExpressLRS in GNU Radio by building upon the existing GNU Radio implementation of LoRa (known as gr-LoRa), to support frequency hopping. ExpressLRS is designed to operate in both 900 MHz and 2.4 GHz. We present performance evaluation results from both software-in-the-loop and hardware-in-the-loop tests based on the ADALM PlutoSDRs and COTS platforms that support the ExpressLRS protocol. We measure the performance of ExpressLRS in the presence of different types of RF interference. The source code for this project is available at:

https://github.com/Diamond-D0gs/GNU_Radio_ExpressLRS.

1. Introduction

LoRaWAN has emerged as a widely adopted and cost-effective Low-Power Wide-Area Network (LPWAN) technology, bridging the gap between short-range wireless protocols and energy-intensive cellular networks. Its low-cost hardware and resilient physical layer make it a cornerstone for Internet of Things (IoT) deployments in agriculture, smart cities, and industrial automation—domains where robust security remains paramount.

ExpressLRS (ELRS) is a modern, open-source radio control link that has gained significant popularity in the FPV (First-Person View) community for its low latency and long-range capabilities. Its resilience to radio frequency (RF) interference is a key feature, attributed to its foundation on LoRa modulation and the implementation of frequency hopping.

In this paper, we describe a functional implementation of ELRS in GNU Radio. We build upon the open-source implementation of the LoRa protocol in GNU Radio and develop custom out-of-tree GNU Radio blocks to support the synchronization and frequency hopping functionalities of ELRS. By emulating key aspects of ELRS in a software-defined radio (SDRs) framework such as GNU Radio, enables repeatable and in-depth performance evaluation of the protocol with off-the-shelf SDRs. Our goal is to investigate performance of ELRS and specifically its resilience against various types of RF interference through both simulation and hardware-in-the-loop testing. This work provides a valuable tool for the radio community to further study, modify, and enhance the ELRS protocol.

2. Background

2.1. LoRa

Originally developed by the French company Cycleo and acquired by Semtech in 2012 (Clarke, 2012), LoRa technology was introduced to the market in 2013 with the SX1272 transceiver (Semtech Corporation, 2013, p. 128). It was designed specifically for the Low-Power Wide-Area Network (LPWAN) segment of the growing IoT market. The protocol enables devices to send small, infrequent amounts of data over very large distances while consuming minimal power, making it ideal for deployments ranging from dense urban sensor networks to sparse rural applications (Semtech Corporation, 2013, p. 1).

LoRa is a flexible physical layer designed to operate in various license-free sub-gigahertz and 2.4 GHz Industrial, Scientific, and Medical (ISM) bands, with frequency allocation varying by region. Transceivers like the SX127x series operate in the sub-gigahertz bands, such as the 433 MHz and the 900 MHz bands (which includes 868 MHz in Eu-

Proceedings of the 15th GNU Radio Conference, Copyright 2025 by the author(s).

rope and 915 MHz in North America). For these bands, LoRa utilizes channel bandwidths of 125 kHz, 250 kHz, and 500 kHz (Semtech Corporation, 2013, p. 10). The newer SX1280 transceiver brought LoRa to the globally available 2.4 GHz ISM band, which supports higher data rates with wider channel bandwidths of 203 kHz, 406 kHz, 812 kHz, and 1625 kHz (Semtech Corporation, 2017, pp. 1, 112).

LoRa achieves its long-range capability by using Chirp Spread Spectrum (CSS), which creates a signal that is more robust to noise and interference. This is done by grouping bits from the data payload into symbols, where each symbol is modulated into a single chirp that sweeps across the signal's bandwidth.

This robustness stems primarily from the coding gain of the chirps, which depends on the Spreading Factor (SF) as the central parameter of LoRa that defines the duration of a chirp and how many bits are encoded into a single symbol and, therefore, the robustness of the transmission. For example, an SF of 8 will generate symbols that encode 8 bits. A higher SF increases the symbol's on-air time, making it easier to distinguish from noise at the cost of a lower data rate, thereby trading speed for range and resilience (Joachim & Burg, 2024; Semtech Corporation, 2013, p. 3).

The `gr-lora_sdr` project (Joachim & Burg, 2024) was created to address the limitations of previous open-source LoRa implementations in GNU Radio. The receiver is compatible with real-world signals by implementing robust algorithms to correct for Carrier Frequency Offset (CFO) and Sampling Time Offset (STO). This improves over prior attempts, which could only demodulate signals with high Signal-to-Noise Ratios (SNRs). As a result, `gr-lora_sdr` is capable of successfully decoding the low-SNR signals typical of the LoRa protocol.

2.2. ExpressLRS Protocol

Prior to the introduction of ExpressLRS, the high-performance, long-range FPV control link market was dominated by proprietary, closed-source systems. Prominent examples included Team BlackSheep's Crossfire (Team BlackSheep, 2024) and FrSky's ACCESS protocol (FrSky Electronic Co., Ltd., 2024), which offers good performance but operates within closed hardware and firmware ecosystems. This landscape created a demand within the FPV community for an open, community-driven, and low-cost alternative that could match or exceed the performance of incumbent technologies.

The ExpressLRS project was introduced with its first public release candidate in April 2021 (ExpressLRS, 2021). As a fully open-source firmware, it offered the FPV community a high-performance alternative to proprietary systems, integrating both remote control and telemetry onto a single radio link. To accommodate different use cases, ExpressLRS supports multiple physical layers. While its long-range modes utilize LoRa combined with a robust frequency hopping scheme, the protocol also features high-rate frequency shift keying (FSK) modes designed to achieve the lowest possible latency for competitive racing (ExpressLRS, 2025c). This paper focuses on the LoRa-based modes, as their spread spectrum design is most relevant to the study of interference resilience. The system was designed to leverage inexpensive and readily available Semtech SX127x/SX1280 hardware, enabling it to achieve significant long-range performance on low-cost transceivers.

ExpressLRS employs a frequency-hopping spread spectrum (FHSS) scheme where the transceiver's center frequency is updated at a configurable, packet-based interval. In the default configuration, this interval is typically set to a small integer, causing the system to transmit multiple packets on a single frequency before hopping to the next channel in the deterministic sequence. The maximum hop interval is limited to sixteen packets, as the value is encoded within a four-bit field in the over-the-air sync packet. Synchronization between the transmitter and receiver is maintained through a unique six-byte UID, which is derived during the initial binding process by applying an MD5 hash to a user-defined binding phrase and using the first six bytes of the resulting hash. This UID serves both to uniquely identify the link upon connection and as the seed for the pseudo-random algorithm that generates the frequency hopping sequence (ExpressLRS, 2025c).

By implementing a Frequency-Hopping Spread Spectrum (FHSS) scheme (ExpressLRS, 2025c), ExpressLRS leverages a well-established technique for mitigating narrowband interference (Goldsmith, 2005). For maximum reliability, the protocol also offers a frequency diversity mode known as Gemini. In this mode, the transmitter sends packets simultaneously and independently on two two frequencies 40 MHz apart for 2.4 GHz and 10 MHz apart for 900 MHz. A true-diversity receiver can then reconstruct the data from whichever link is more clear, providing a powerful defense against band-specific interference or signal fading (ExpressLRS, 2025a). The scope of this study is limited to the more common single-transceiver configuration and does not evaluate the performance of Gemini.

To achieve low latency, ExpressLRS employs several key strategies. First, it uses small data packets, with payloads for control data typically ranging from 8 to 13 bytes. Second, in its high-performance modes, it utilizes low LoRa SF to minimize on-air time (ExpressLRS, 2025c). Finally, the protocol multiplexes control data by prioritizing critical information. The primary flight controls, such as joystick

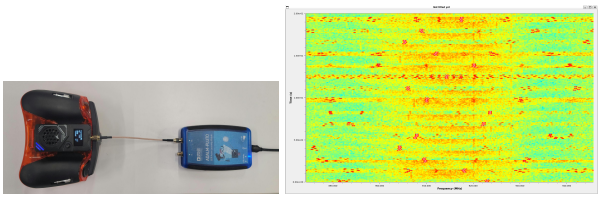
inputs and arming status, are transmitted in every packet. Non-critical auxiliary channels are then sent in a round-robin fashion, with each packet carrying the critical data plus the next auxiliary input in the sequence (ExpressLRS, 2025b).

ExpressLRS can be configured to support various packet rates, channel counts, and channel resolutions to meet different performance goals. The most common configuration, known as "Hybrid" mode, multiplexes switch data to prioritize low latency. By switch data we refers to the physical switches on the radio transmitter that can be mapped to different functions in the flight controller. In this mode, critical flight controls like joystick inputs are sent in every packet for the fastest possible response. The remaining auxiliary channels (i.e., extra control channels beyond the main four stick channels throttle, roll, pitch, yaw) are then sent one at a time in a round-robin sequence. This approach guarantees minimal delay for essential controls while ensuring all other functions are still updated reliably, offering a significant advantage over modes that send all channels at once at the cost of higher latency (ExpressLRS, 2025b).

The ExpressLRS protocol is designed for global operation and complies with diverse regional spectrum regulations through a system of configurable "domains". Each domain defines a specific set of RF parameters, including the start and stop frequencies of the hopping band, the total number of frequency channels, and a nominal center frequency. These presets ensure that the transmitter operates legally within the Industrial, Scientific, and Medical (ISM) bands allocated by different regulatory bodies. For example, the `FCC915` domain, configured for the United States, utilizes 40 frequency channels across a 23.4 MHz-wide band centered at 915 MHz (ExpressLRS, 2025c).

To meet stricter regional requirements, such as those mandated by ETSI for the European `EU868` domain, ExpressLRS incorporates a Listen Before Talk (LBT) mechanism for medium access control. Prior to a scheduled transmission, the firmware uses the LoRa transceiver's channel activity detection (CAD) mode to measure the received signal strength indicator (RSSI) on the target frequency (Semtech Corporation, 2013). If the channel is detected as busy, the transmission for that packet interval is aborted, and the system proceeds to the next frequency in its hopping sequence for the subsequent packet. This integration of LBT with FHSS ensures regulatory compliance by avoiding transmission on occupied channels while leveraging frequency diversity to maintain link reliability in congested environments.

Synchronization is maintained between two transceivers by having the ExpressLRS transmitter from one transceiver initiate transmission of synchronization packets as part of its standard FHSS sequence. The synchronization packet



(a) (b)

Figure 1. Physical capture of an ExpressLRS transmitter during its synchronization phase. (a) A COTS ExpressLRS transmitter configured to the FCC915 domain connected to an ADALM-Pluto SDR. (b) The spectrogram shows the frequency-hopping beacon packets as the transmitter searches for a connection.

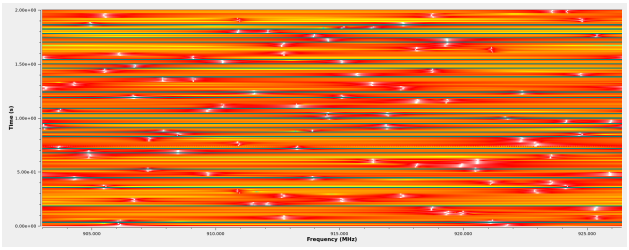


Figure 2. Spectrogram of the composite signal generated from five simulated ELRS transmitters running concurrently with 40 frequency channels and SF of 7.

contains critical data for the receiver to lock on, including the transmitter's current index in the FHSS sequence, the current position in the auxiliary channel round-robin sequence, the last two bytes of the truncated binding phrase hash (UID), and other configuration information. Upon boot or when a connection is lost or when the transmitter detects that he is not receiving a telemetry packet from its corresponding receiver for a pre-defined time period the transmitter enters a search phase. In this phase, the transmitter sends synchronization packets at a significantly increased rate across its hopping pattern in an attempt to quickly re-establish the connection (ExpressLRS, 2025c).

To provide a visual illustration of the protocol's behavior, a signal from a COTS 915 MHz ExpressLRS transmitter was captured using an ADALM-Pluto SDR. Figure 1 depicts the physical hardware (left) and the IQ sample-level spectrogram (right) generated by the captured ELRS transmissions during the link establishment phase.

3. ExpressLRS in GNU Radio

To adapt the existing `gr-lora_sdr` framework for ExpressLRS, our work required modifications to the data handling between blocks. The original Whitening and Dewhitening blocks were rewritten to pass data as byte arrays

rather than their original string-based message payloads. This change was a necessary prerequisite for integrating the ExpressLRS state machine, allowing the transmission and reception of serialized packet data. We developed an ELRS loopback flowgraph, as shown in Figure 3, that contains the modified blocks along with new OOT blocks in GNU Radio to test and evaluate ELRS physical layer's performance. It is important to note that this flowgraph models key aspects of the ExpressLRS protocol, namely its packet timing and FHSS capabilities, rather than implementing the full protocol state machine.

3.1. Transmitter GNU Radio Flowgraph

The transmitter side of the flowgraph (top left) emulates a simplified ExpressLRS data stream. A `Message Strobe` block generates a trigger every 20 ms (emulating a 50 Hz packet rate). This trigger is sent to the custom `Counter Formatter` block. This block's primary role is to generate the payload for each packet to be transmitted. Upon receiving a trigger, it formats an internal counter into a 6-byte string, padded with leading zeros to five characters and terminated with a newline character. It then outputs this string as a GNU Radio message and increments its internal counter. This process is repeated for a user-defined number of packets, creating a predictable sequence that can be easily verified at the receiver to calculate packet error rates. This specific payload was chosen to ensure a consistent 6-byte packet length for LoRa transmission, a common size for ELRS data packets. The user can define how many packets will be sent up to a limit of 99,999. The formatted string is then passed to a standard `LoRa TX` hierarchical block from the `gr-lora_sdr` project. This block handles all physical layer processing, including whitening, encoding, interleaving, and modulation, and adds LoRa's standard 16-bit CRC for integrity checks.

A custom `FHSS Controller` block generates the frequency hopping sequence based on a user-defined binding phrase. This block is an implementation of the ExpressLRS FHSS algorithm. During initialization, it takes the binding phrase, frequency band parameters (start, stop, and frequency channel count), and derives a 6-byte unique ID (UID) by applying an MD5 hash. This UID is then used to seed a 16-bit linear congruential generator (LCG) with the same constants (0x343FD and 0x269EC3) used in the official firmware. The LCG is then used to perform a Fisher-Yates shuffle on an array of channel indices, creating the deterministic, pseudo-random hopping sequence. During operation, the FHSS Controller block receives the same trigger as the `Counter Formatter`. For every other packet, it advances its position in the pre-calculated frequency sequence and outputs a message containing the corresponding frequency offset. This offset is used by a `Signal Source` and a `Multiply` block to dynamically shift the center frequency of the modulated LoRa signal after it has been generated, thus simulating the frequency hop.

The `FHSS Controller` block is designed to be highly configurable, allowing its behavior to be defined through several key parameters exposed in GNU Radio. These parameters directly correspond to the regulatory domains defined by the ExpressLRS protocol. The user provides a `binding_phrase` string, which is used as the primary input to the MD5 hash function to generate the unique seed for the pseudo-random hopping sequence. The physical characteristics of the hopping band are defined by `freq_start` and `freq_stop` (the lowest and highest frequencies in Hz), the total `freq_count` of channels, and the nominal `freq_center` of the band. Finally, a boolean `disable` flag is provided for testing purposes. When set to true, this flag bypasses the hopping algorithm and forces the block to output a zero offset, keeping the transmitter on the center frequency.

3.2. Receiver GNU Radio Flowgraph

The receiver side of the flowgraph in Fig. 3 (right) is designed to de-hop and decode the simulated signal. The incoming wideband signal, containing the frequency-hopped packets, is first multiplied with a signal from a conjugate `Signal Source` that is fed by the same `FHSS Controller` logic. This mixing operation reverses the frequency shift applied at the transmitter, effectively stabilizing the signal at a constant center frequency. Following the de-hopping, a `Rational Resampler` block downsamples the signal to the appropriate rate for the `LoRa Rx` block. The `LoRa Rx` block, also from `gr-lora_sdr`, then performs the complete LoRa demodulation and decoding process. Finally, the decoded payload from the `LoRa Rx` block is passed to a `File Sink`. The resulting file, containing the received packet numbers, can then be analyzed offline to calculate packet error rates by examining the payloads missing in the generated file.

4. Experimental Setup

We use GNU Radio version 3.10.1.1 for our experiments. The LoRa physical layer is implemented using the `gr-lora_sdr` project (Joachim & Burg, 2024).

4.1. Software-in-the-Loop (SITL) Simulation

To evaluate resilience of ERLS to co-channel interference, a series of SITL simulations were conducted using the GNU Radio flowgraph described in Section 3. In this setup, multiple independent ELRS transmitters were simulated concurrently, with their outputs summed to create a composite signal that was then fed to a single receiver chain.

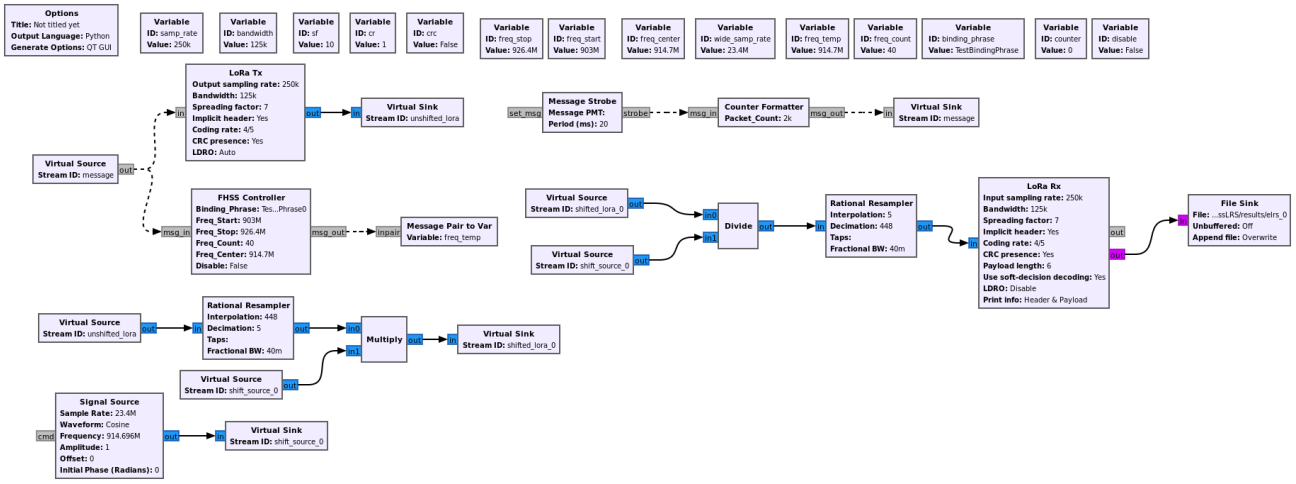


Figure 3. GNU Radio flowgraph for Software-in-the-Loop simulation of an ExpressLRS transmitter and receiver.

This methodology emulates a real-world scenario where a single link may be operating in the presence of multi-user interference generated by multiple co-located transmitters.

For all SITL experiments, LoRa sample rate is set to 250 KSps, LoRa bandwidth is 125 KHz, SF is set to 7, coding rate is $4/5$, and resampled bandwidth and sample rate is 23.4 MHz. To validate interference robustness in multiple access scenarios, we vary the number of interfering transmitters, the number of available frequency channels, and the SF. The number of concurrent transmitters is varied from 1 to 5. We test 10, 20, 30, and 40 frequency channel counts. For the tests evaluating the impact of SF, the number of frequency channels is fixed to 40 while we vary SF from 7 to 10. Packet error rate (PER) performance of the desired transmitter is measured by counting the total number of lost packets over the transmission of 5000 packets. Packet loss is determined by analyzing the output of the receiver's file sink and identifying missing sequence numbers in the received payloads.

4.2. Hardware-in-the-Loop (HITL) Testing

To validate the functionality of the implementation on physical SDR hardware, we conduct a Hardware-in-the-Loop (HITL) test. This test is designed as a proof-of-concept to demonstrate that the GNU Radio flowgraph successfully controls an SDR for both transmission and reception, rather than to capture detailed performance metrics. The setup utilizes a low-cost Analog Devices ADALM-Pluto SDR in a wired, full-duplex loopback configuration, where the device's Tx output is connected directly to its Rx input. The corresponding GNU Radio flowgraph is depicted in Figure 4.

The signal flow mirrors the SITL simulation, with the virtual sources and sinks replaced by their hardware counterparts. The transmitter chain, consisting of the `LoRa Tx` and `FHSS Controller`, generates a FHSS baseband signal. This complex signal is then sent to the `PlutoSDR Sink` block, which modulates it to the specified 914.7 MHz carrier center frequency.

In parallel, the `PlutoSDR Source` on the same device (identified by the same IP address) receives the over-the-wire RF signal. The receiver chain first de-hops the signal by mixing it with the output from its synchronized `FHSS Controller`. The resulting baseband signal is then resampled and passed to the `LoRa Rx` block for demodulation and decoding. Successful reception of the packet sequence in this loopback test confirmed that the flowgraph can correctly interface with and control the SDR hardware for both transmission and reception of FHSS LoRa signals.

5. Performance Evaluation

5.1. Software-in-the-Loop (SITL)

5.1.1. Varying Frequency Channel Counts

The results from varying frequency channel counts are presented in Fig. 5. Data show a direct correlation between the number of available frequency channels and ELRS resilience to co-channel interference. In test configurations with only 10 or 20 channels, we observe an increase in packet error rate as the number of concurrent Tx increases from 3 to 5. The 20-channel configuration, for instance, peaked at an average loss of 2.2 packets for a total of 5000 transmitted packets. Increasing the number of available channels to 30 and 40 significantly improves performance,

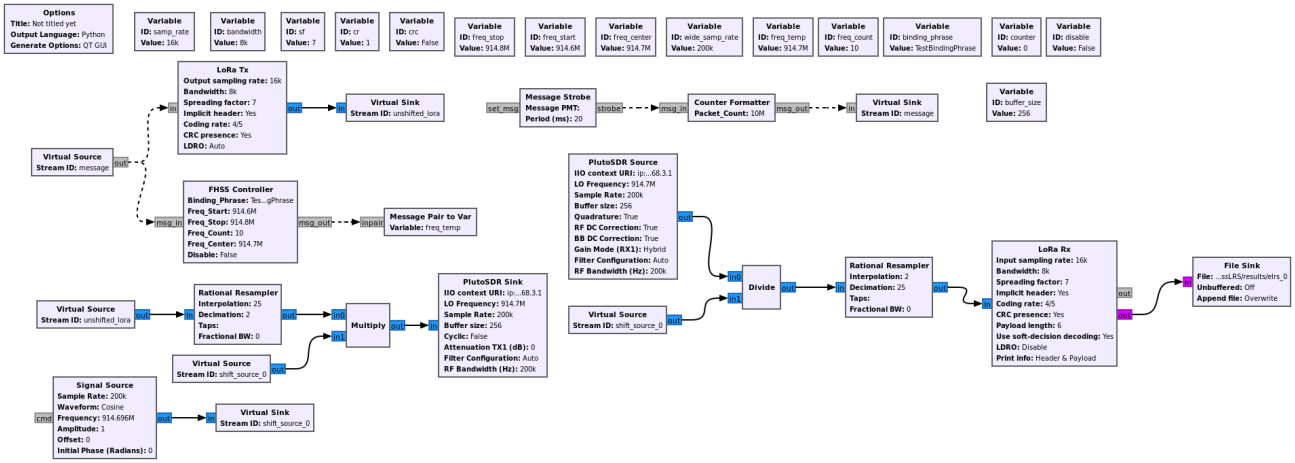


Figure 4. GNU Radio flowgraph for the Hardware-in-the-Loop test, using a single ADALM-Pluto SDR in a loopback configuration.

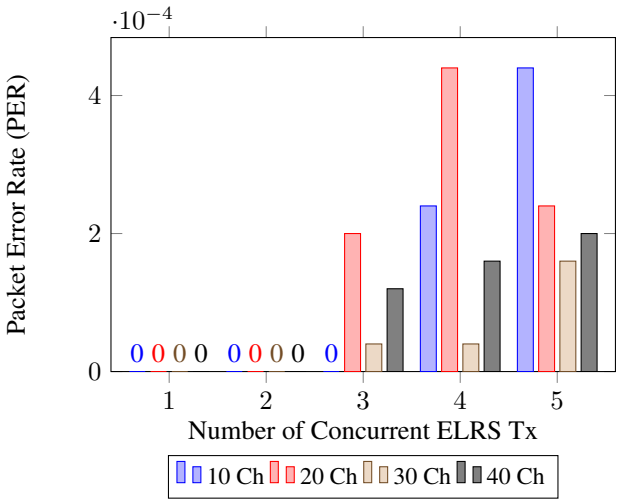


Figure 5. PER vs. number of concurrent transmitters, when varying frequency channel counts across 5000 packet transmissions.

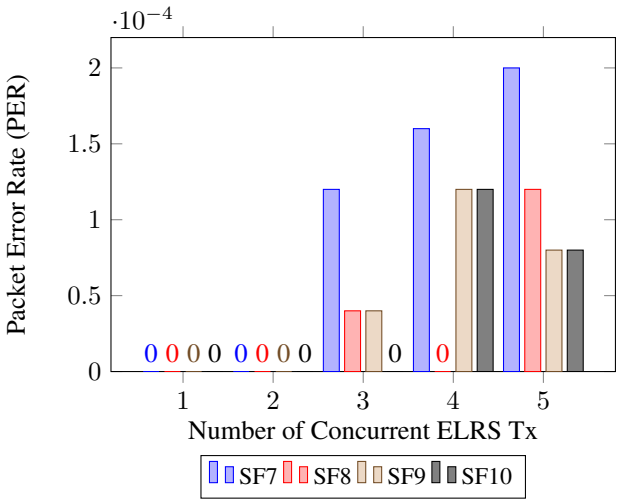


Figure 6. PER vs. number of concurrent transmitters, when varying SF across 5000 packet transmissions.

with the 40-channel test maintaining an average packet loss of only 1 packet, even with 5 concurrent Tx. We demonstrate that a larger frequency pool is critical in mitigating multi-user interference in dense FHSS environments.

5.1.2. Varying SF

Figure 6 illustrates the impact of LoRa's SF on interference resilience. The number of frequency channels is kept fixed to 40 to mirror ELRS `FCC915`. PER results indicate that a higher SF provides a performance advantage in the presence of multiple interference sources. While all configurations performed well with 1 or 2 concurrent transmitters, the SF7 option shows a progressive increase in PER as the number of Tx increases from 3 to 5. Conversely, SF8, SF9, and SF10 configurations exhibited substantially greater resilience, maintaining an average packet loss between 0.4 and 0.6 packets in the most congested scenario. This finding aligns with the theoretical properties of spread spectrum communications, where a higher SF increases processing gain and makes the signal more robust to in-band interference (Goldsmith, 2005).

5.1.3. Varying Concurrent Links

Figure 7 depicts PER when the number of concurrent Tx increases from 5 to 10 and 15, while SF is set to 7 and we use 40 frequency channels. As expected PER increases as co-

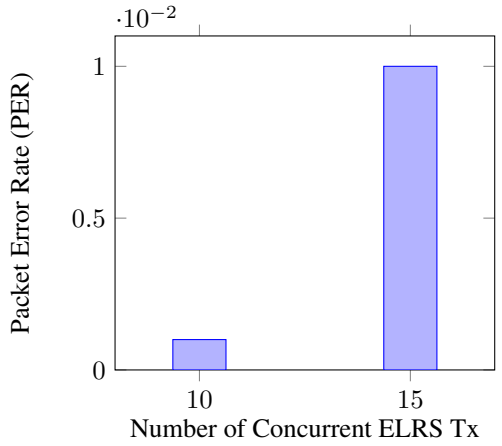


Figure 7. PER when varying the number of ELRS Tx for 1000 packet transmissions.

channel interfers increase. When attempting to simulate a higher number of interfering sources (e.g., 20 transmitters), the increased computational load on the host machine led to significant timing jitter in the scheduler. This jitter caused the asynchronous message containing the frequency hop information to become misaligned with the corresponding packet's sample stream at the receiver. As the de-hopping message no longer arrived at the correct time, the receiver's demodulation process would fail. Therefore, the practical limit for this specific hardware and software configuration was determined to be approximately 15 concurrent Tx.

5.2. Hardware-in-the-Loop (HITL) Challenges

In the HITL environment, the unavoidable buffering on both the transmit (sink) and receive (source) ends of the SDR introduced a variable and unpredictable delay. To achieve fast frequency hopping required by ELRS, the asynchronous message which triggers frequency change must be time-aligned with the corresponding packet's sample stream. This latency resulted in a systemic misalignment between the frequency de-hopping and the received samples at the Rx. Ongoing work explore how to use gr-IIO OOT block to align data passed to the Pluto Sink block with the selected LO frequency, while also exploring other SDR devices from the USRP family.

6. Conclusion and Future Work

In this paper, we present the first open-source GNU Radio implementation of the FHSS mechanism of the ExpressLRS protocol. By building upon the existing `gr-lora_sdr` physical layer, we created a self-contained simulation framework capable of analyzing ELRS performance in the presence of co-channel interference. Our SITL performance evaluation provided a quantitative baseline, demonstrating that packet error rate scales predictably with an increase in the number of concurrent transmitters and a decrease in the number of available frequency channels. The results also confirmed that higher SF improve the link's robustness in congested spectrum environments.

Limitations of our current implementation is a performance ceiling in the SITL simulation at approximately 15 concurrent transmitters, caused by timing jitter in the GNU Radio scheduler that prevented reliable synchronization between frequency hopping commands and their corresponding data packets. Furthermore, HITL tests are challenged by unpredictable latencies from SDR sample buffering, which affects the required timing precision for the FHSS de-hopping mechanism.

Future work will focus on extending the capabilities of our flowgraphs to implement more advanced features of ExpressLRS such as Gemini's frequency diversity mode, which would allow for a quantitative analysis of ELRS resilience to band-specific jamming, as well as the protocol's ability to transport MAVLink telemetry for semi-autonomous systems. Given the timing-critical nature of these features, future development may need to move the simulation to a platform that supports discrete-event simulation, such as MATLAB, while still leveraging the proven `gr-lora_sdr` module for the generation of the physical layer LoRa waveforms. For higher HITL fidelity testing, moving the entire implementation to the FPGA of the SDR will eliminate software-based timing issues and enable true real-time performance analysis and inter-operation with COTS ELRS chipsets.

6.1. Software and Data

Our GNU Radio flowgraphs and source code for reproducing our experiments are available open-source at:

`https://github.com/Diamond-D0gs/GNU_Radio_ExpressLRS`

References

Clarke, Peter. Semtech buys French IP developer. *EE Times*, Mar 2012. [Online]. Available: `https://www.eetimes.com/semtech-buys-french-ip-developer/`.

ExpressLRS. ExpressLRS Version 1.0.0-RC2, Apr 2021. [Online]. Available: `https://github.com/ExpressLRS/ExpressLRS/releases/tag/1.0.0-RC2`.

ExpressLRS. ExpressLRS Documentation: Gemini. `https://www.expresslrs.org/software/gemini/`, 2025a. Accessed: September 1, 2025.

ExpressLRS. ExpressLRS Documentation: Switch Configuration. https://www.expresslrs.org/software/switch-config/, 2025b. Accessed: September 1, 2025.

ExpressLRS. ExpressLRS Firmware Source Code. https://github.com/ExpressLRS/ExpressLRS, 2025c. Accessed: September 1, 2025.

FrSky Electronic Co., Ltd. ACCESS Protocol, 2024. [Online]. Available: https://www.frsky-rc.com/frsky-advanced-communication-control\-elevated-spread-spectrum-access-\protocol/.

Goldsmith, Andrea. *Wireless Communications*. Cambridge University Press, 2005.

Joachim, Tapparel and Burg, Andreas. Design and implementation of lora physical layer in gnu radio. *Proceedings of the GNU Radio Conference*, 9(1), 2024.

Semtech Corporation. *SX1272/73 - 860 MHz to 1020 MHz Low Power Long Range Transceiver*. Semtech Corporation, Jun 2013. Revision 1. [Online]. Available: https://www.semtech.com/products/wireless-rf/lora-transceivers/sx1272.

Semtech Corporation. *SX1280/SX1281 Long Range, Low Power, 2.4 GHz Transceiver with Ranging Capability*. Semtech Corporation, May 2017. Data Sheet, Rev 1.1, DS.SX1280-1.W.APP. [Online]. Available: https://www.semtech.com/products/wireless-rf/lora-transceivers/sx1280.

Team BlackSheep. TBS Crossfire Long Range Radio System, 2024. [Online]. Available: https://www.team-blacksheep.com/products/prod:crossfire_tx.